

Junior Verification Engineer / Junior UVM Role — Preparation Guide

This preparation guide is intended for candidates interviewing for a junior-level Design Verification role with emphasis on SystemVerilog, UVM, functional coverage, assertions, protocols, and basic computer architecture.

1. Core UVM Concepts

The candidate should understand the purpose of UVM and how it is used to build reusable, scalable verification environments.

UVM Factory

The candidate should be able to explain the UVM factory and why it is useful.

Important topics:

- Factory registration using ``uvm_object_utils` and ``uvm_component_utils`
- Object creation using `type_id::create()`
- Difference between direct construction using `new()` and factory-based construction
- Factory override use cases

The candidate should know both types of factory overrides:

```
set_type_override_by_type(original_type::get_type(),  
    override_type::get_type());
```

and

```
set_inst_override_by_type(  
    original_type::get_type(),  
    override_type::get_type(),  
    "env.agent.driver"  
);
```

Expected understanding:

- Type override replaces all instances of a given type.
- Instance override replaces only a specific hierarchical instance.
- Factory override is commonly used to swap drivers, sequences, monitors, or transaction types without changing the original testbench source code.

2. UVM Config DB

The candidate should understand how `uvm_config_db` is used to pass configuration information hierarchically.

Important topics:

- `set()` and `get()`
- Hierarchical paths
- Passing virtual interfaces from the top module into the UVM environment
- Passing agent configuration, active/passive mode, address ranges, or protocol settings
- Debugging failed `get()` calls

Example:

```
uvm_config_db#(virtual axi_if)::set(
    null,
    "uvm_test_top.env.agent.*",
    "vif",
    axi_vif
);
```

```
if (!uvm_config_db#(virtual axi_if)::get(
    this,
    "",
    "vif",
    vif
)) begin
    `uvm_fatal("NOVIF", "Virtual interface not found")
end
```

The candidate should know that `uvm_config_db` is built on top of the UVM resource database. If time permits, the candidate can also read about `uvm_resource_db`, but `uvm_config_db` is more important for junior-level preparation.

3. UVM Components

The candidate should know the role of common UVM components.

Driver

The driver converts transaction-level sequence items into pin-level activity on the DUT interface.

The candidate should understand:

- How a driver gets items from a sequencer
- `seq_item_port.get_next_item(req)`
- Driving signals through a virtual interface

- Calling `seq_item_port.item_done()`

Sequencer

The sequencer arbitrates between sequences and provides sequence items to the driver.

The candidate should know the relationship between:

- Sequence
- Sequence item
- Sequencer
- Driver

Monitor

The monitor observes DUT interface signals and converts pin-level activity back into transaction-level objects.

The candidate should understand:

- Passive observation
- Use of analysis ports
- Sending observed transactions to scoreboard, coverage collector, or predictor

Scoreboard

The scoreboard checks actual DUT behavior against expected behavior.

Important concepts:

- Expected vs actual comparison
- In-order and out-of-order checking
- Use of queues, associative arrays, or FIFOs
- Transaction comparison

Predictor

A predictor generates expected transactions based on input stimulus or observed input-side transactions.

The candidate should understand that a predictor is commonly used when expected behavior can be modeled at transaction level.

Subscriber / Coverage Collector

A subscriber receives transactions through an analysis export and is commonly used for functional coverage collection.

The candidate should understand:

```

class my_cov extends uvm_subscriber #(my_txn);
  `uvm_component_utils(my_cov)

  my_txn tx;

  covergroup cg;
    coverpoint tx.addr;
    coverpoint tx.kind;
    cross tx.addr, tx.kind;
  endgroup

  function void write(my_txn t);
    tx = t;
    cg.sample();
  endfunction
endclass

```

4. Functional Coverage — Very Important

Functional coverage is a key interview topic.

The candidate should clearly understand the difference between functional coverage and code coverage.

Functional Coverage

Functional coverage is user-defined coverage. It answers the question:

Did we test the intended design features and scenarios?

Important topics:

- Covergroups
- Coverpoints
- Bins
- Illegal bins
- Ignore bins
- Cross coverage
- Sampling
- Coverage closure

Example:

```

covergroup axi_cg;
  coverpoint burst_type {
    bins fixed = {0};
    bins incr  = {1};
    bins wrap  = {2};
  }

```

```

coverpoint burst_len {
    bins short = {[1:4]};
    bins long  = {[5:16]};
}

cross burst_type, burst_len;
endgroup

```

The candidate should be able to explain:

- What feature is being covered
- When the covergroup is sampled
- Why cross coverage is needed
- How functional coverage helps identify missing scenarios

Code Coverage

Code coverage is tool-generated coverage. It answers the question:

Which parts of RTL code were exercised during simulation?

Important code coverage types:

- Line coverage
- Statement coverage
- Branch coverage
- Toggle coverage
- FSM coverage
- Expression coverage

The candidate should understand that high code coverage does not guarantee functional correctness. Functional coverage is needed to ensure meaningful scenarios were verified.

5. TLM — Transaction Level Modeling

The candidate should understand transaction-level communication between UVM components.

Important topics:

- `uvm_analysis_port`
- `uvm_analysis_export`
- `uvm_analysis_imp`
- Blocking TLM ports
- Nonblocking TLM ports
- Put/get/peek style communication
- Analysis communication for broadcast-style transactions

Sequence Item vs Sequence

The candidate must know the difference.

A sequence item represents one transaction.

Example:

```
class axi_item extends uvm_sequence_item;
  rand bit [31:0] addr;
  rand bit [31:0] data;
  rand bit        write;

  `uvm_object_utils(axi_item)

  function new(string name = "axi_item");
    super.new(name);
  endfunction
endclass
```

A sequence generates one or more sequence items.

Example:

```
class axi_write_seq extends uvm_sequence #(axi_item);
  `uvm_object_utils(axi_write_seq)

  function new(string name = "axi_write_seq");
    super.new(name);
  endfunction

  task body();
    axi_item req;

    req = axi_item::type_id::create("req");
    start_item(req);
    assert(req.randomize() with { write == 1; });
    finish_item(req);
  endtask
endclass
```

Expected understanding:

- Sequence item = transaction object
- Sequence = stimulus generator
- Sequencer = arbitrates and sends sequence items to driver
- Driver = converts sequence items to signal activity

6. Constraint Randomization

The candidate should understand SystemVerilog randomization basics.

Important topics:

- `rand`
- `randc`
- Constraint blocks
- Inline constraints
- Constraint solver
- Soft constraints
- `pre_randomize()` and `post_randomize()`

Example:

```
class packet;
  rand bit [7:0] addr;
  rand bit [3:0] len;

  constraint c_len {
    len inside {[1:8]};
  }

  constraint c_addr {
    addr[1:0] == 2'b00;
  }
endclass
```

Soft constraint example:

```
constraint default_len {
  soft len == 4;
}
```

The candidate should know that a soft constraint can be overridden by a stronger inline or hard constraint.

7. SystemVerilog Language Topics

The candidate should be comfortable with core SystemVerilog constructs.

Important topics:

- `logic`, `bit`, `reg`, `wire`
- Packed vs unpacked arrays
- Dynamic arrays
- Associative arrays

- Queues
- Structs and enums
- Tasks and functions
- Interfaces
- Clocking blocks
- Modports
- Program block basics, if applicable
- `always_comb`, `always_ff`, `always_latch`
- Blocking vs nonblocking assignments
- ``timescale`

8. Fork / Join and Threading

The candidate should understand parallel execution in SystemVerilog.

Important constructs:

```
fork
    task1();
    task2();
join
```

```
fork
    task1();
    task2();
join_any
```

```
fork
    task1();
    task2();
join_none
```

Expected understanding:

- `join` waits for all threads
- `join_any` waits for any one thread to finish
- `join_none` continues without waiting
- `disable fork` can be used to terminate forked threads
- Fork/join is commonly used in testbenches for parallel stimulus, timeout checks, monitors, and protocol checkers

9. Object-Oriented Programming

The candidate should understand OOP basics because UVM is class-based.

Important topics:

- Class
- Object
- Handle
- Constructor
- Inheritance
- Polymorphism
- Encapsulation
- Virtual methods
- Method overriding
- Superclass and subclass
- Type casting

Example:

```
class parent;
  int a, b, c;

  function int add();
    return a + b + c;
  endfunction
endclass

class child extends parent;
  function int sub();
    return a - b;
  endfunction
endclass
```

Important handle behavior:

```
parent p1;
child c1;

p1 = new(); // p1 points to a parent object
c1 = new(); // c1 points to a child object
```

This downcast is invalid if `p1` points to a pure parent object:

```
$cast(c1, p1); // Fails if p1 does not actually point to a child object
```

This works only if the parent handle points to a child object:

```
child c2;
parent p2;
```

```

c2 = new();
p2 = c2;           // Upcast: valid
if (!$cast(c1, p2)) // Downcast: valid because p2 points to child object
    $error("Cast failed");

```

The candidate should clearly know:

- Upcasting from child to parent is allowed.
- Downcasting from parent to child requires `$cast`.
- `$cast` succeeds only if the actual object type is compatible with the target type.

10. Interfaces, Clocking Blocks, and Modports

The candidate should understand why interfaces are used in verification.

Important topics:

- Grouping related signals
- Avoiding long port lists
- Sharing signals between DUT and testbench
- Virtual interface usage in UVM
- Clocking blocks
- Modports
- Input/output skew
- Race-condition avoidance

Example:

```

interface axi_if(input logic clk);
    logic valid;
    logic ready;
    logic [31:0] data;

    clocking drv_cb @(posedge clk);
        output valid;
        output data;
        input ready;
    endclocking

    clocking mon_cb @(posedge clk);
        input valid;
        input ready;
        input data;
    endclocking

    modport DRV (clocking drv_cb);
    modport MON (clocking mon_cb);
endinterface

```

Expected understanding:

- Driver uses a clocking block to drive signals at the correct time.
- Monitor uses a clocking block to sample signals consistently.
- Clocking block skew helps avoid race conditions between DUT and testbench.
- Modports define signal direction and access restrictions.

11. SystemVerilog Assertions

The candidate should know basic SVA concepts.

Important topics:

- Immediate assertions
- Concurrent assertions
- Properties
- Sequences
- Implication operators: `| ->` and `| =>`
- Liveness vs safety properties
- Protocol checking
- Arbitration checks
- Request-grant checks

Example arbitration property:

```
property req_gets_grant;  
  @(posedge clk)  
  disable iff (!rst_n)  
  req | -> ##[1:5] grant;  
endproperty  
  
assert property (req_gets_grant);
```

The candidate should understand that this checks whether a request is eventually served within 1 to 5 cycles.

Recommended reference:

- Verification Academy SVA Cookbook
- Basic arbiter assertion examples
- Handshake protocol assertions

12. Protocol Knowledge

The candidate should have basic protocol awareness. Deep protocol expertise is not expected for a junior role, but the candidate should know common signals and transaction flow.

On-Chip Protocols

Important protocols:

- AMBA APB
- AMBA AHB
- AMBA AXI
- AHB-to-APB bridge basics

For AXI, the candidate should know:

- Read address channel
- Read data channel
- Write address channel
- Write data channel
- Write response channel
- Valid-ready handshake
- Burst length
- Burst type
- ID
- Protection signal
- Cache signal
- Lock signal
- QoS, if applicable

For APB, the candidate should know:

- PADDR
- PWDATA
- PRDATA
- PSEL
- PENABLE
- PWRITE
- PREADY
- PSLVERR

Expected APB flow:

1. Setup phase: PSEL = 1, PENABLE = 0
2. Access phase: PSEL = 1, PENABLE = 1
3. Transfer completes when PREADY = 1

Off-Chip Protocols

Important protocols:

- UART
- SPI
- I2C
- USB basics

SPI Basics

The candidate should know basic SPI signals:

- **MOSI** — Master Out, Slave In
- **MISO** — Master In, Slave Out
- **SCLK** — Serial clock
- **SS** / **CS_N** — Slave select / chip select

The candidate should understand:

- SPI is usually full-duplex.
- Master generates the clock.
- Slave select chooses the target slave.
- CPOL and CPHA define clock polarity and phase.
- Data is shifted on clock edges.

13. Computer Architecture Basics

The candidate should understand basic architecture tradeoffs.

Important topics:

- Power
- Performance
- Area
- Cost
- Latency
- Throughput
- On-chip vs off-chip implementation tradeoffs

Power

The candidate should know dynamic power at a conceptual level:

Dynamic power depends on switching activity, capacitance, voltage, and frequency.

Typical formula:

$$P_{\text{dynamic}} \approx \alpha C V^2 f$$

Where:

- **α** = switching activity factor
- **C** = capacitance
- **V** = voltage
- **f** = frequency

The candidate should understand that reducing switching activity, voltage, frequency, or capacitance can reduce dynamic power.

Pipelining

The candidate should know the classic 5-stage pipeline:

1. Instruction Fetch
2. Instruction Decode
3. Execute
4. Memory Access
5. Write Back

Important hazards:

- Data hazard
- Control hazard
- Structural hazard

The candidate should understand:

- Forwarding/bypassing can reduce data hazards.
- Stalling may be required when data is not ready.
- Branch prediction helps reduce control hazard penalty.
- Pipelining improves throughput but not necessarily single-instruction latency.

14. UVM vs Formal Verification

The candidate should understand the difference between simulation-based verification and formal verification.

UVM / Simulation-Based Verification

UVM is dynamic and time-based.

Characteristics:

- Uses simulation
- Applies random or directed stimulus
- Checks DUT behavior over time
- Uses scoreboard, assertions, and coverage
- Good for system-level and protocol-level verification
- Can run long regressions with many randomized seeds

Limitations:

- Cannot exhaustively test all possible states
- Bugs may be missed if stimulus does not hit the scenario
- Coverage closure is required to measure progress

Formal Verification

Formal verification is static and mathematically exhaustive within a bounded or constrained state space.

Characteristics:

- Uses assertions/properties
- Explores possible states mathematically
- Can prove or disprove properties
- Good for control logic, arbiters, FIFOs, deadlock checks, and protocol rules

Limitations:

- State-space explosion
- May require black boxing
- Over-constraint can hide bugs
- Under-constraint can produce unrealistic failures
- Complex datapath-heavy designs may be difficult to prove fully

The candidate should be able to explain:

- UVM asks: "Did my simulation test this scenario?"
- Formal asks: "Can this property ever fail?"
- UVM is stimulus-driven.
- Formal is property-driven.

15. Suggested Interview Preparation Priority

The candidate should prioritize preparation in this order:

1. SystemVerilog basics
2. OOP concepts
3. UVM component architecture
4. Sequence item, sequence, sequencer, and driver flow
5. Config DB
6. Factory and overrides
7. Functional coverage
8. Code coverage
9. Interfaces, modports, and clocking blocks
10. Assertions
11. TLM communication
12. Constraint randomization
13. Common protocols: APB, AXI basics, SPI
14. Computer architecture basics
15. UVM vs formal verification

16. Expected Coding Practice

The candidate should be able to write or explain small examples for:

- A simple transaction class

- A sequence item with constraints
- A sequence generating randomized items
- A driver using `get_next_item()` and `item_done()`
- A monitor using an analysis port
- A subscriber with a covergroup
- A scoreboard receiving expected and actual transactions
- A config DB set/get example
- A factory override example
- A clocking block and modport example
- A simple concurrent assertion
- A fork/join example
- A type-casting example using `$cast`

17. Strong Junior-Level Expectation

For a junior role, the candidate does not need to be an expert in all areas, but should be able to:

- Explain basic UVM architecture clearly
- Understand how stimulus moves from sequence to driver to DUT
- Understand how monitors, scoreboards, and coverage collectors observe and check behavior
- Write basic SystemVerilog class-based code
- Understand functional coverage and why it is important
- Explain basic assertions
- Understand simple protocols such as APB and SPI
- Show awareness of AXI channel structure and valid-ready handshaking
- Explain the difference between simulation and formal verification
- Discuss debugging logically and systematically

The most important areas for interview focus are:

- SystemVerilog OOP
- UVM sequence/driver/monitor/scoreboard flow
- Config DB
- Factory override
- Functional coverage
- Code coverage
- Interfaces and clocking blocks
- Assertions
- Basic protocol knowledge